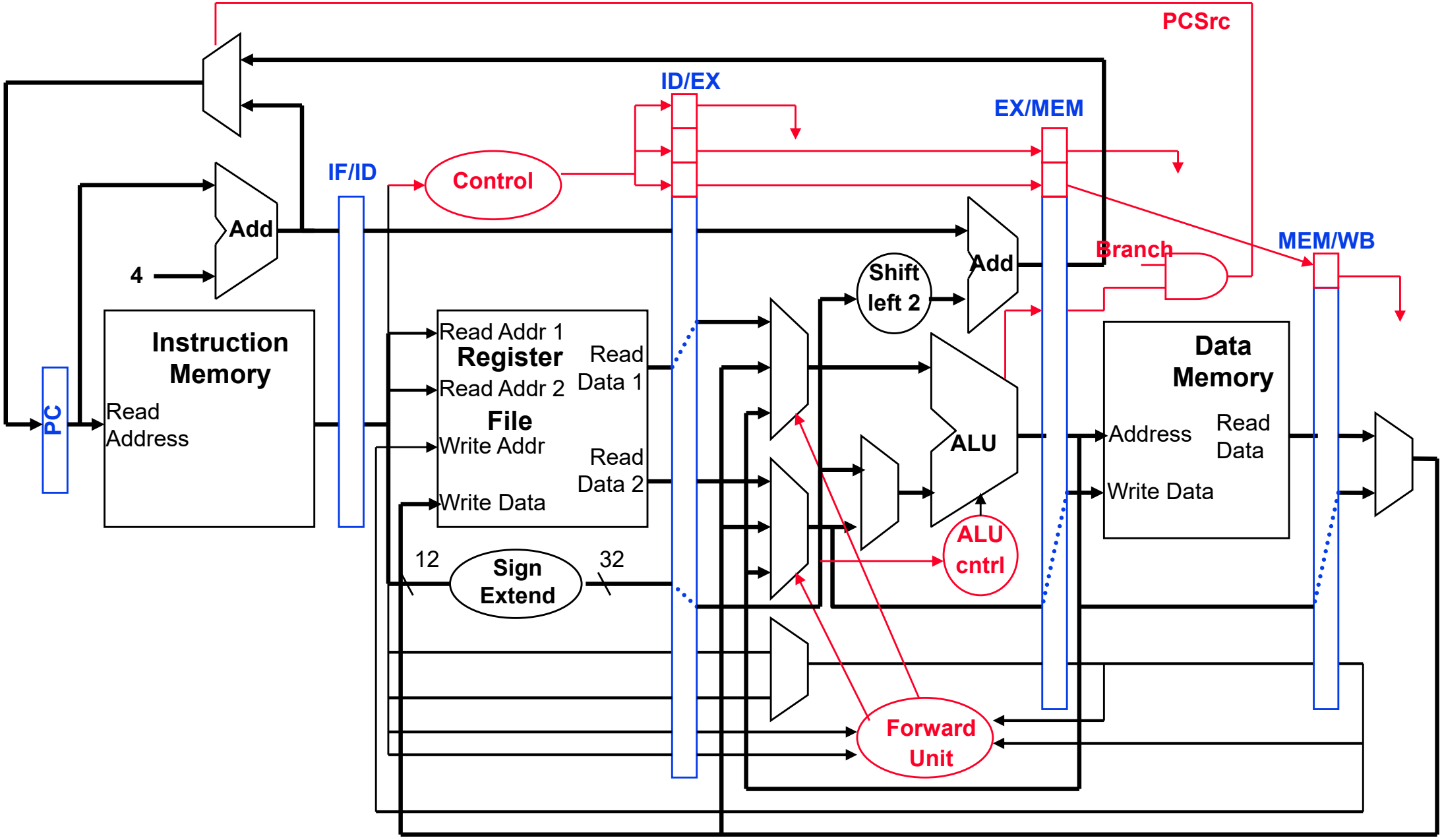
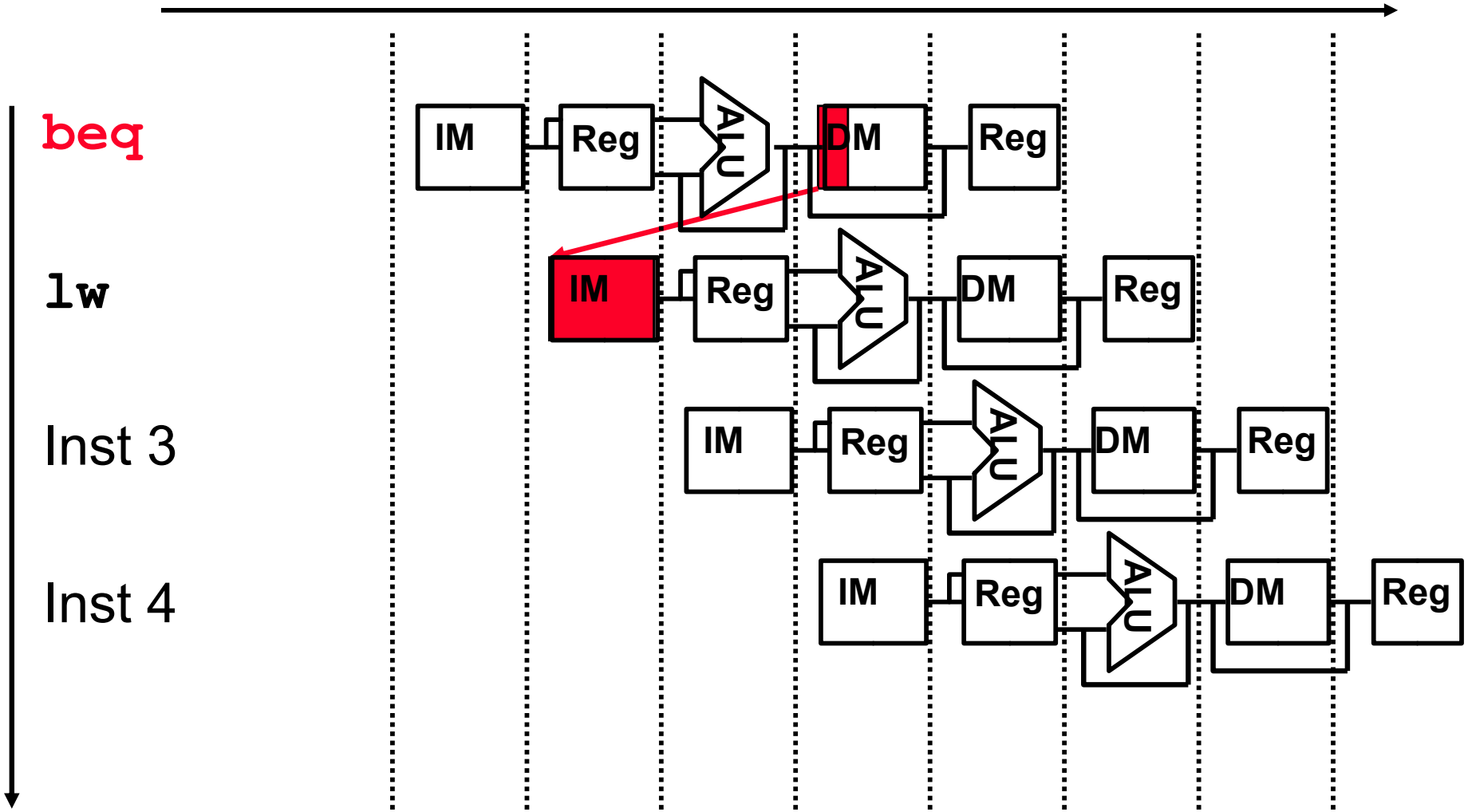
Datapath per istruzione branch



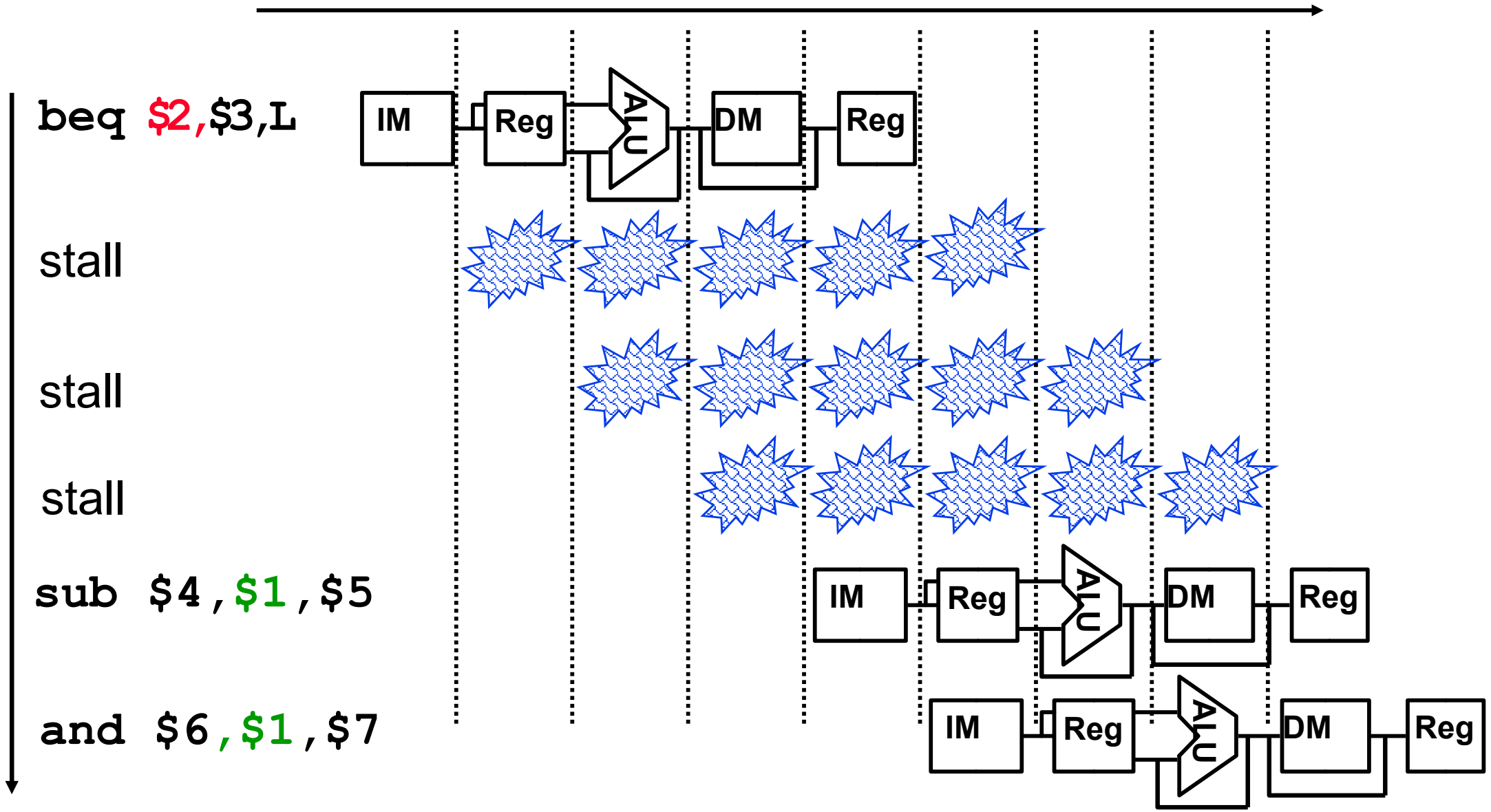
Conflitti su controllo di flusso

- Istruzioni coinvolte:
  - Branch condizionate/incondizionate
  - Exceptions
- Possibili soluzioni:
  - Stallo (aumenta CPI)
  - Anticipare la decisione
  - Ritardare la decisione (necessario support dal compilatore)
  - Previsione e correzione

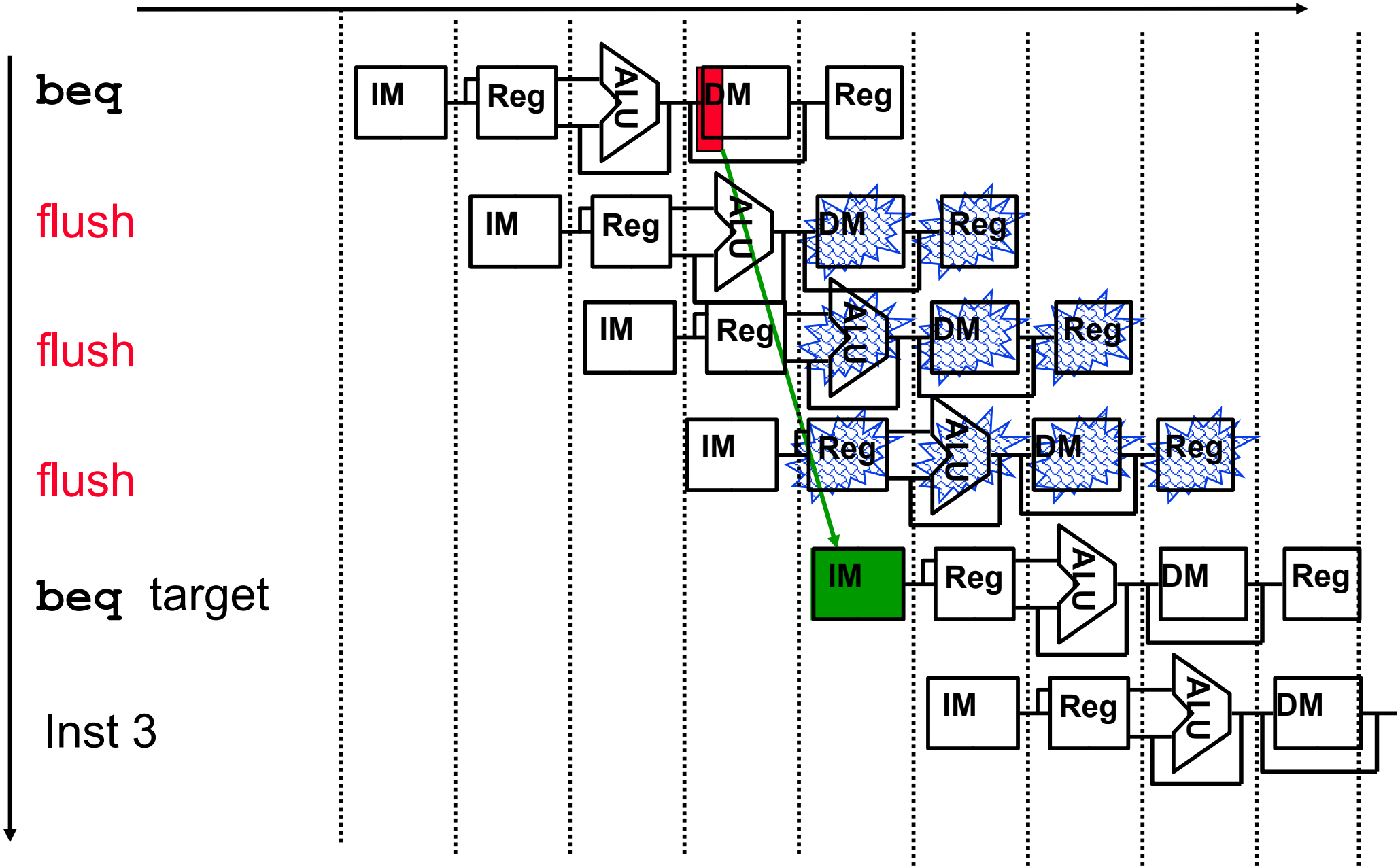
Branch



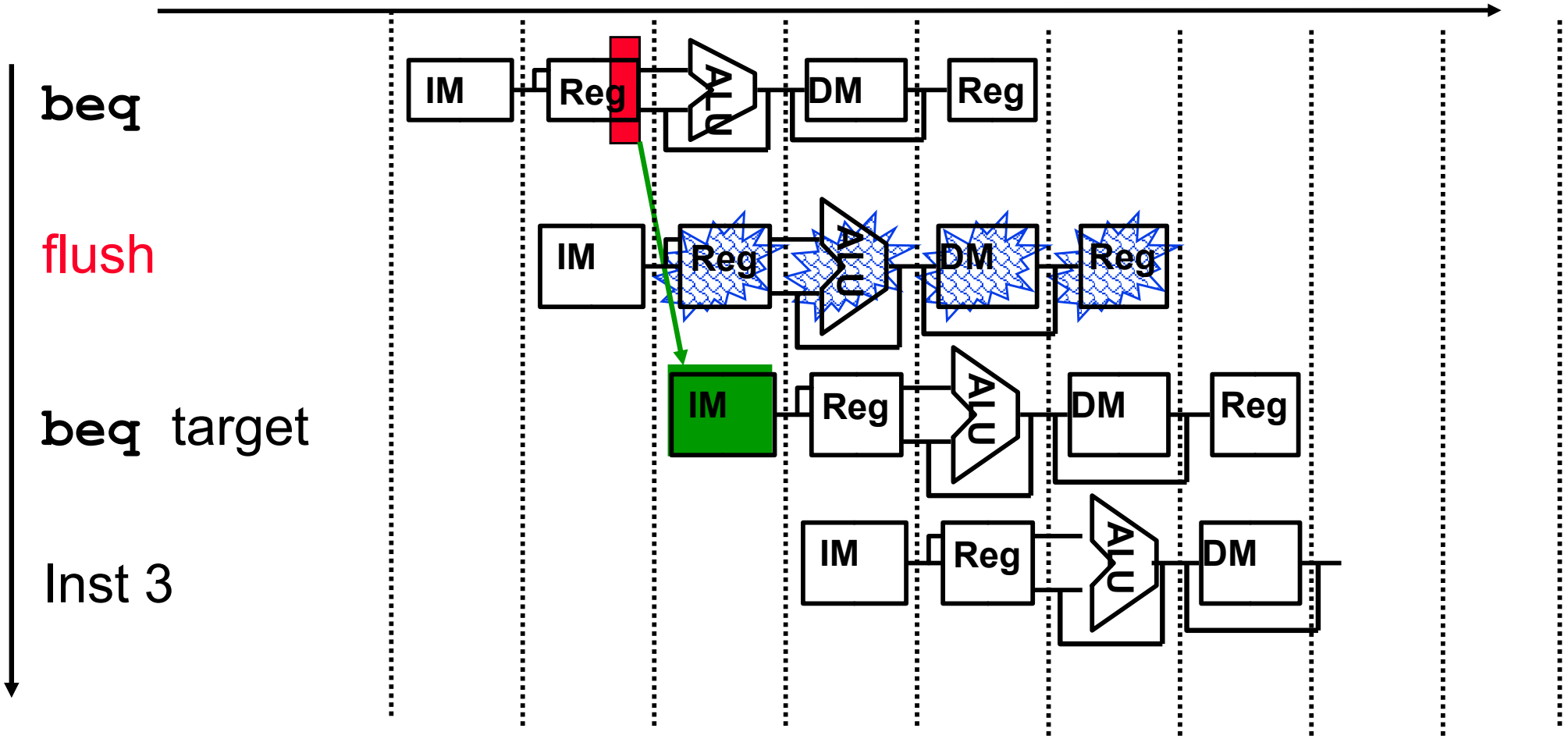
Come evitare Control Hazard



Soluzione semplice

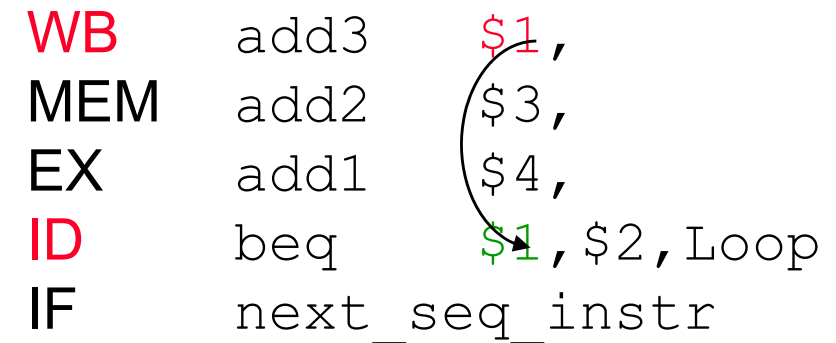


Possibile anticipare la decisione ?

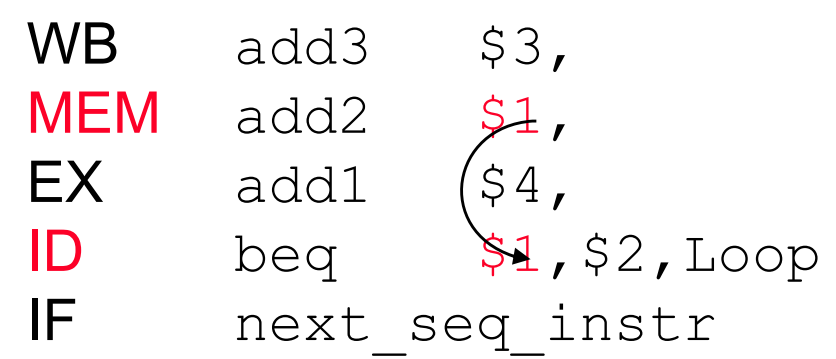


ID Branch Forwarding : possibili problemi

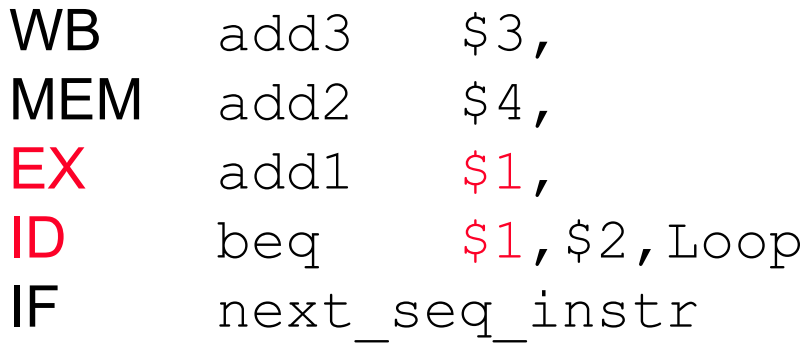
- WB



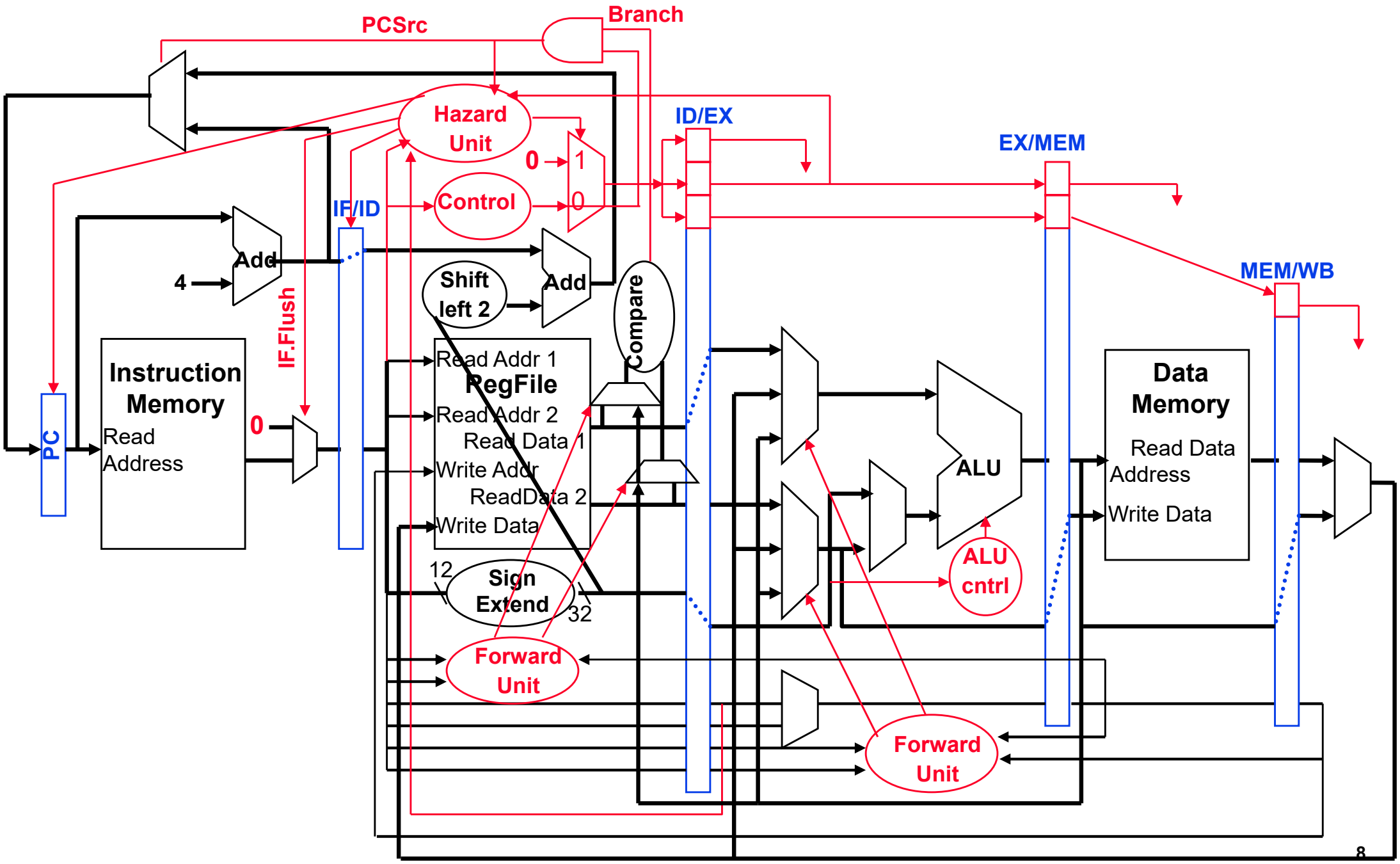
- MEM



- EX richiede stallo aggiuntivo

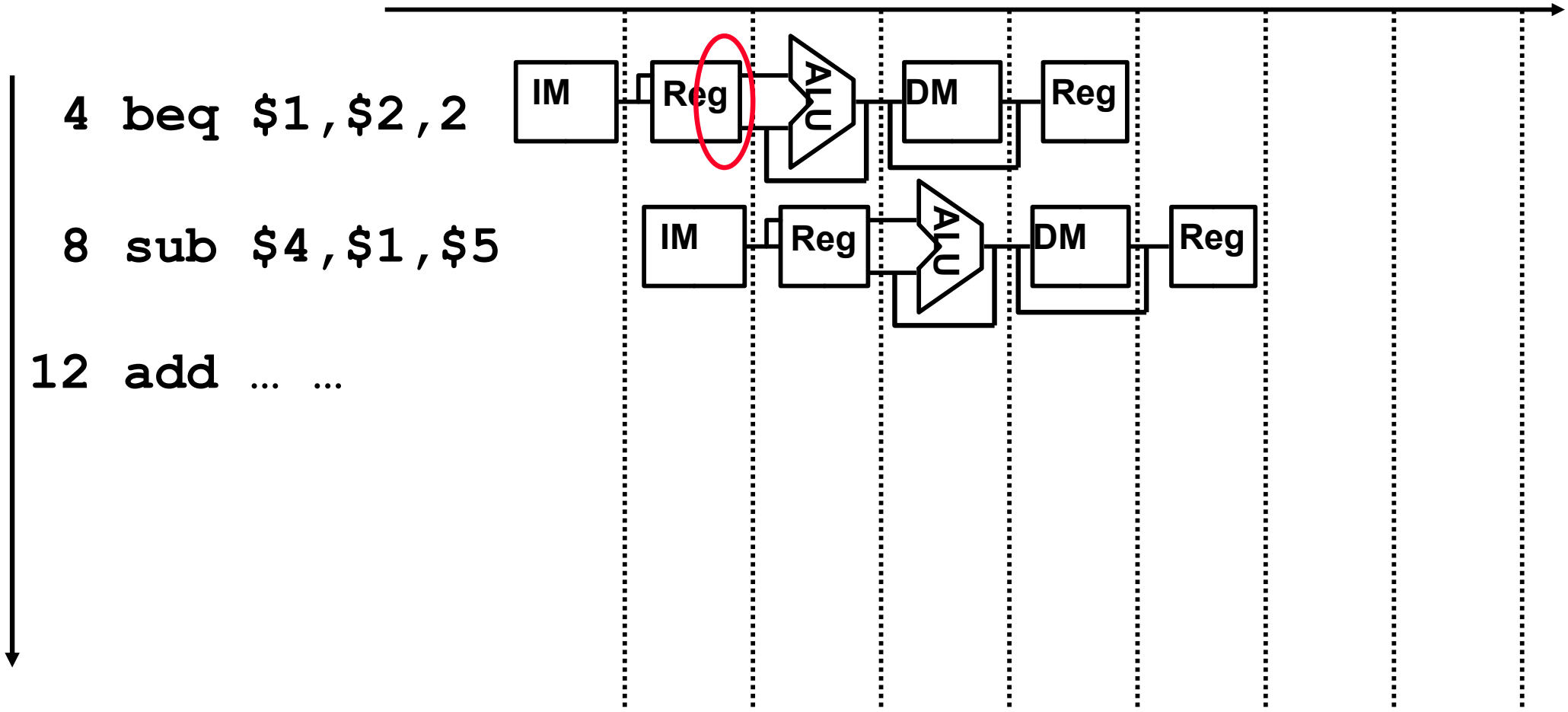


Supporto per salti condizionati nella fase ID



Branch prediction statica

Previsione esito del salto fissata a sempre eseguito o viceversa sempre non eseguito.



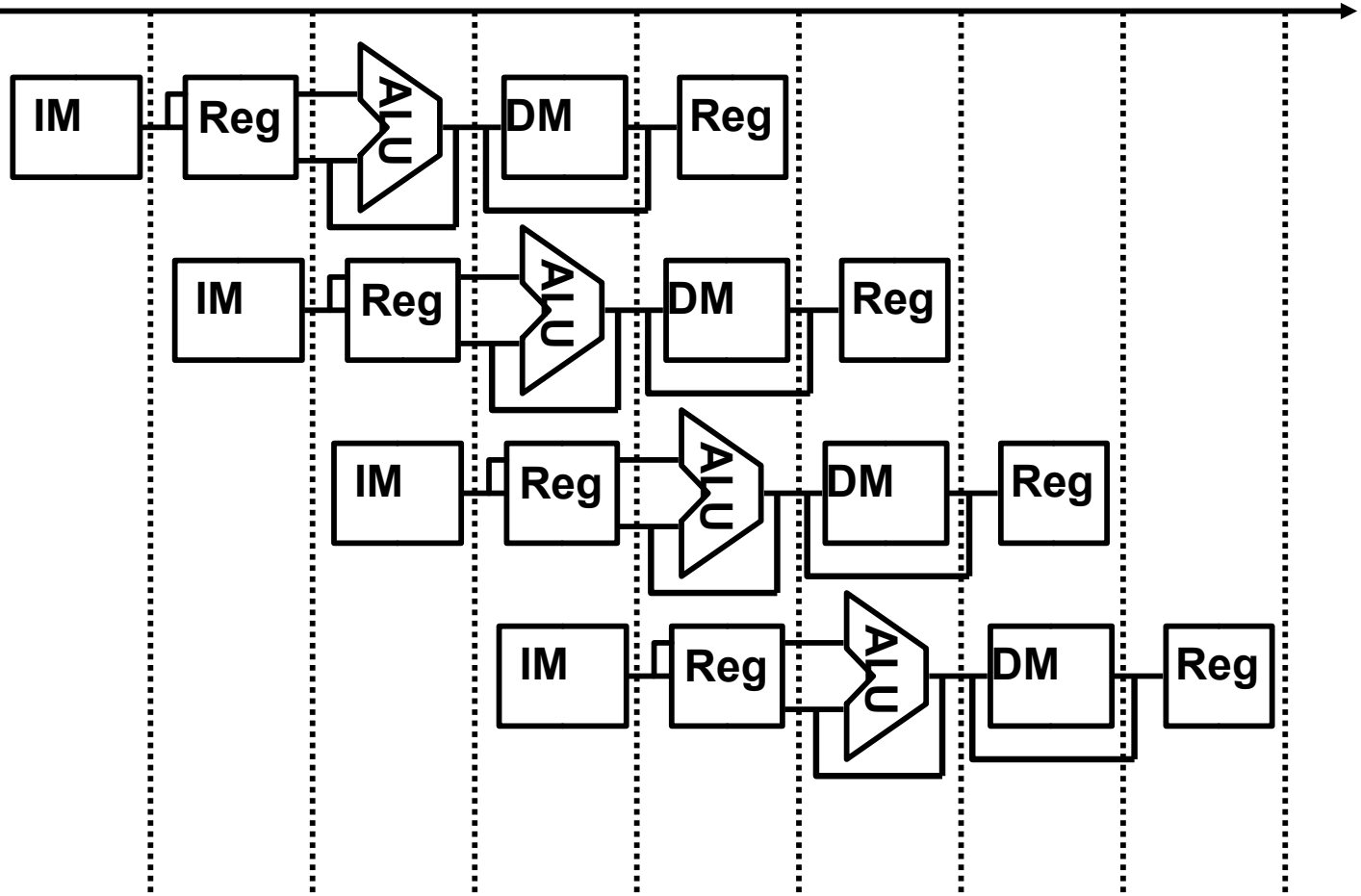
Flushing a causa di previsione errata (Not Taken)

4 beq $1,$2,2

8 sub $4,$1,$5

36 and $6,$1,$7

40 or r8,$1,$9



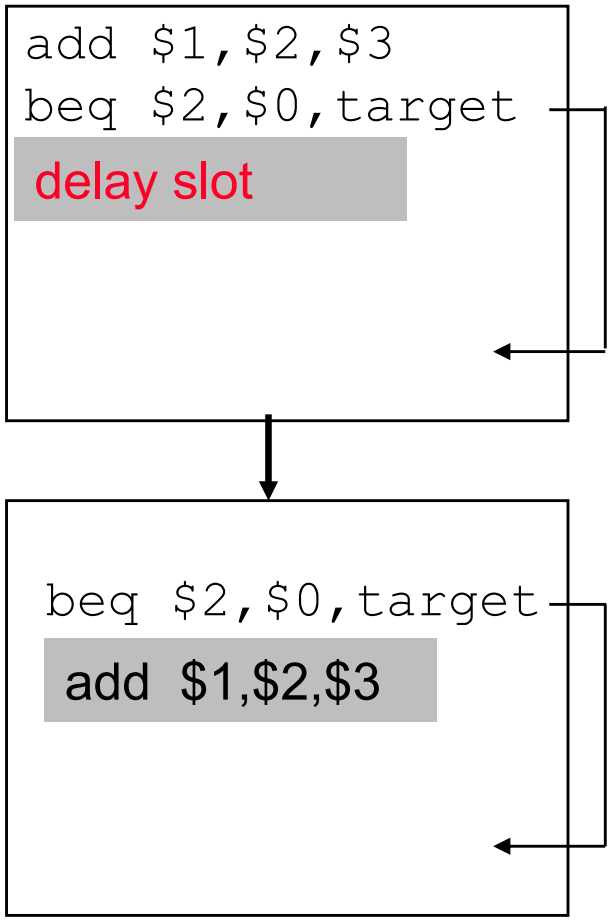
Ottimizzazione : Delayed Branches

- delayed branches = eseguire sempre la successiva istruzione sequenziale : di fatto l'eventuale salto viene effettuato dopo l'istruzione successiva.

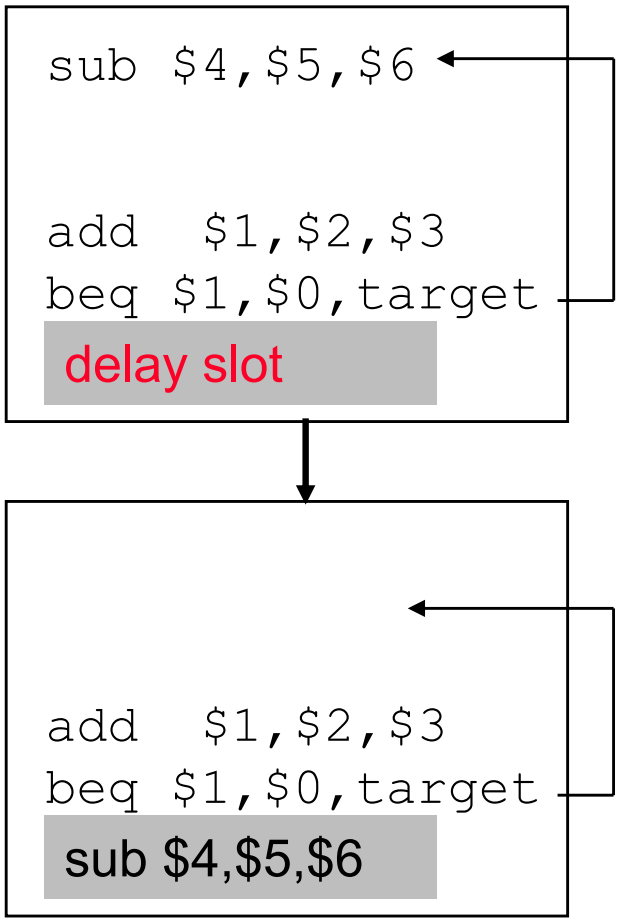
  Il compilatore pone dopo una istruzione branch una istruzione che non dipende dall'esito del salto condizionato. Lo stallo necessario a causa della branch viene così "nascosto" ovvero sfruttato per eseguire comunque un'istruzione .

Scheduling delle Branch Delay Slots

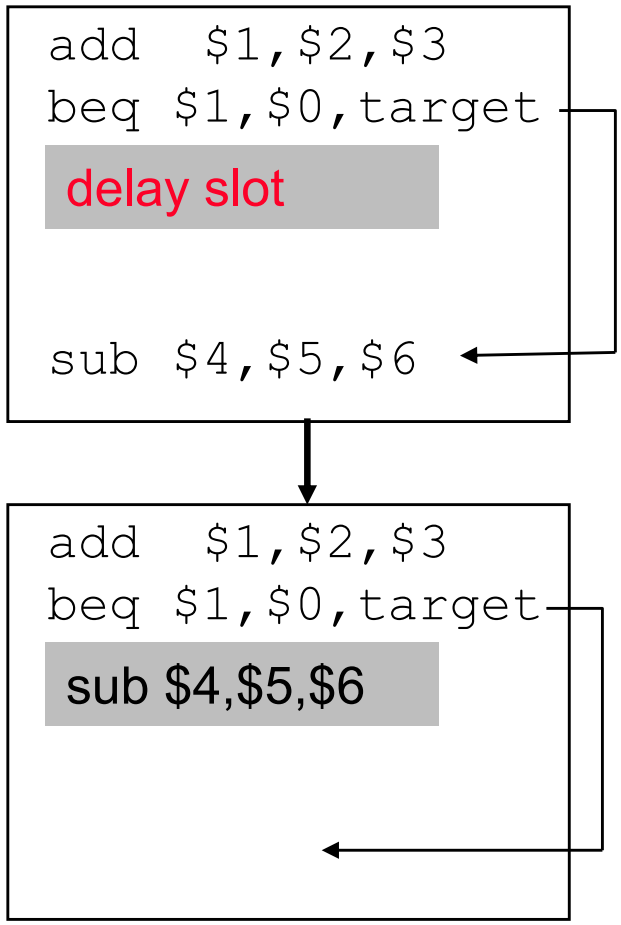
A. Istruzione precedente



B. Istruzione all'indirizzo di salto indietro



C. Istruzione all'indirizzo di salto avanti



Uso Branch prediction statica

- Previsione “not taken” funziona bene nelle strutture di salto “top of the loop”

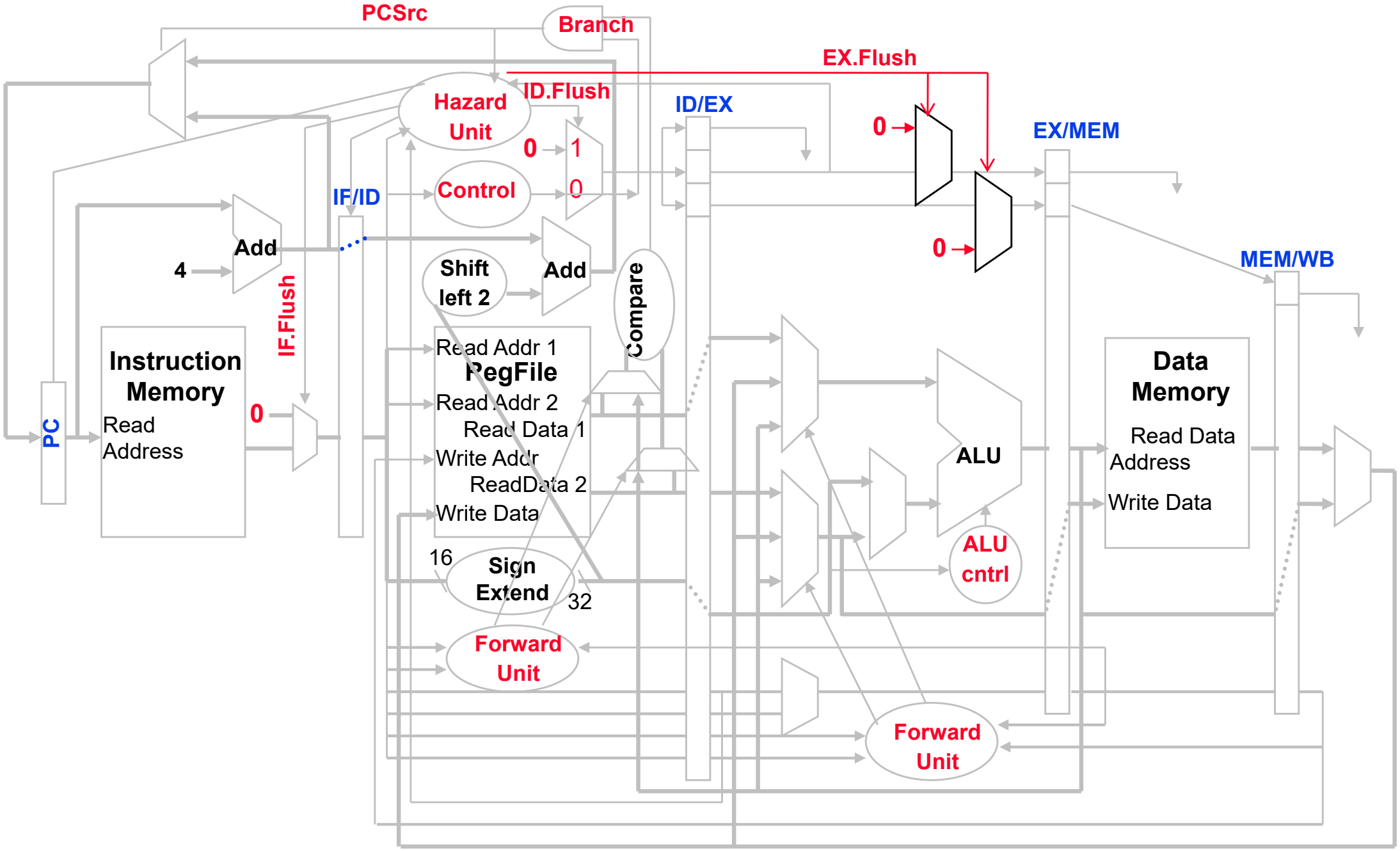
  Rimane tuttavia una istruzione jump

```
Loop: beq $1,$2,Out
      add ……
            .
            .
            .
      sub ……..
      j   Loop
Out:  lw ……..
```

- Previsione “not taken" NON funziona bene per strutture di salto “bottom of the loop”

```
Loop: add ……
      lw ……
            .
            .
            .
      sub ……
      bne $1,$2,Loop
      lw……
```

Aggiunta branch prediction



Ancora su branch prediction

- Man mano che la penalità causata dalle istruzioni branch aumenta (per pipeline più profonde), un semplice schema di previsione statico danneggerà le prestazioni.
- E' possibile provare a prevedere l'esito della esecuzioni di istruzioni branch in modo dinamico durante l'esecuzione del programma :questo richiede una CU più sofisticata e complessa.
- Dynamic branch prediction: prevedere esito delle istruzioni branch durante l'esecuzione dei programmi utilizzando informazioni derivanti dalla precedente esecuzione delle stesse istruzioni di salto.

Branch Prediction dinamica

- Utilizza un branch prediction buffer anche noto come branch history table (BHT):

  La tabella è usata nello stadio IF utilizzando come indirizzo i bit meno significativi del PC e contiene l'esito riscontrato in precedenza di una istruzione branch.

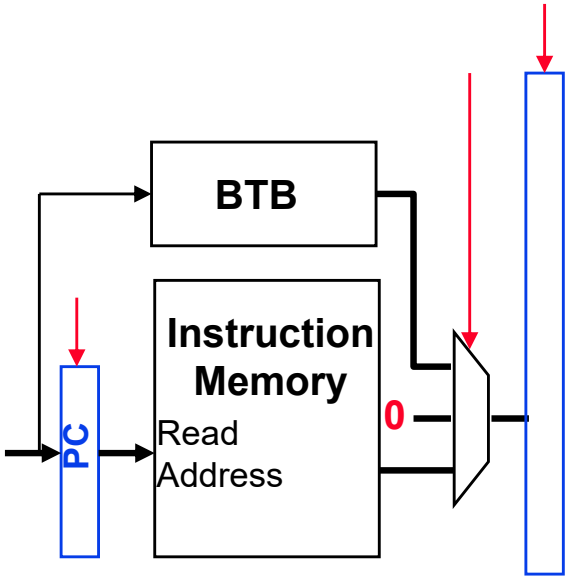
  Se la predizione risulta errata si ritorna a stallo+flush ma si cambia anche il bit (i bits) nella BHT.

  La percentuale di previsioni errate può variare dall'1% al 20%:

  1% nasa7, tomcat
  18% eqntott
  12% gcc

BHT e Branch Target Buffer

- La BHT consente di prevedere *se* l'esito di una istruzione branch è "taken", ma non dice dove saltare
- Un branch target buffer (BTB) salva gli indirizzi "branch target" oppure direttamente le istruzioni (già lette in precedenza).



Predizione a 1 bit

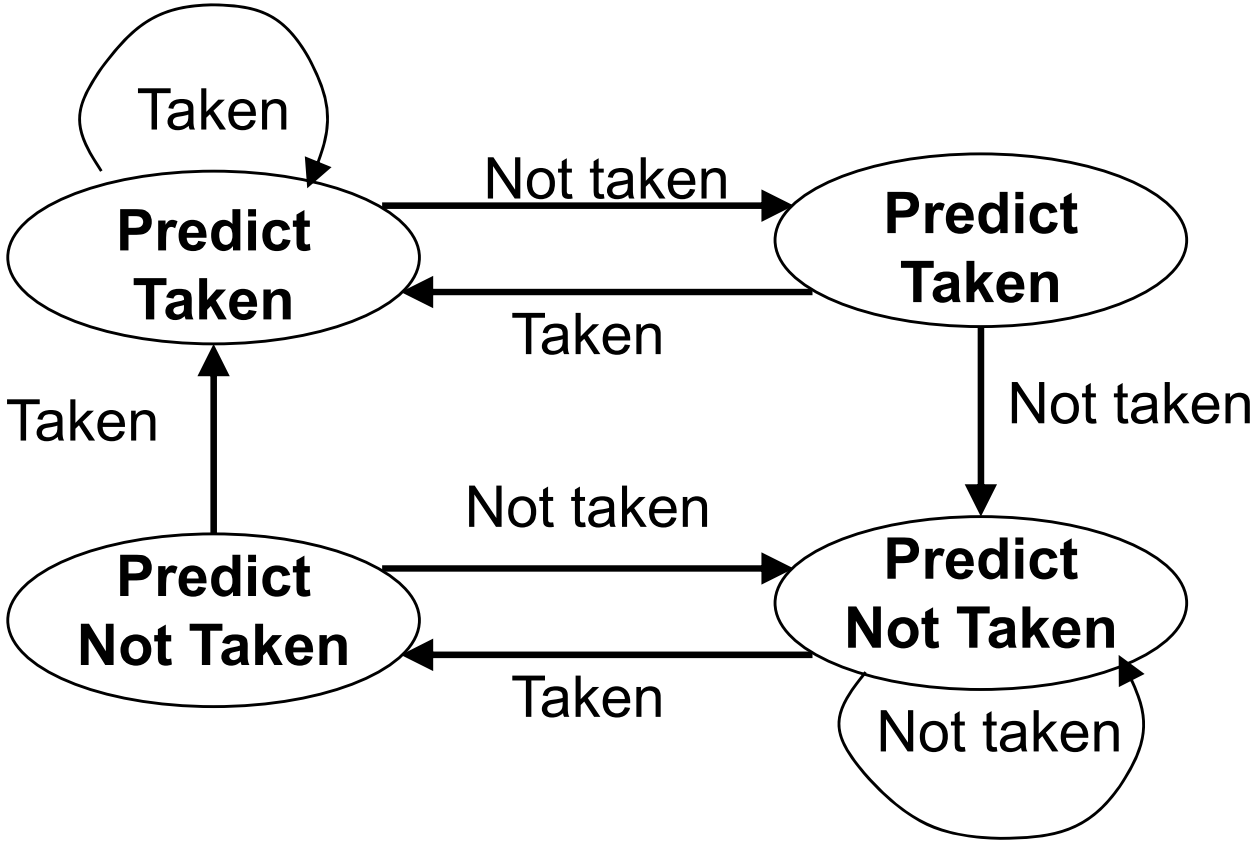
- Esempio : previsione nel caso di loop

1. bit = 0 all'inizio
2. Prima previsione sbagliata (predict_bit = 1)
3. Le successive sono corrette
4. L'ultima, in uscita è quindi sbagliata (predict_bit = 0)
5. Se loop ha 10 ripetizioni 80% corrette

```
Loop: 1st loop instr
      2nd loop instr
            .
            .
            .
      last loop instr
      bne $1,$2,Loop
      fall out instr
```

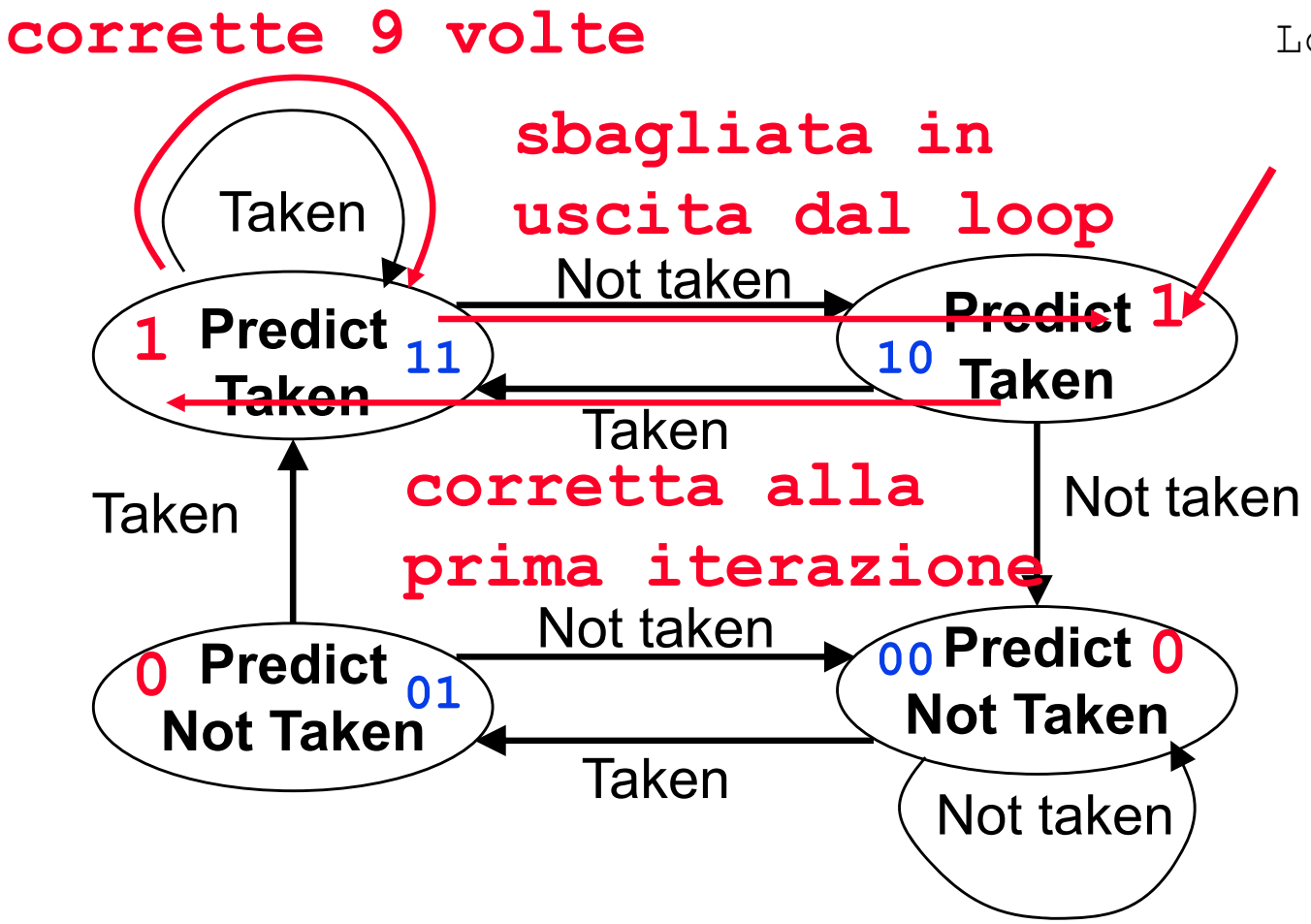
Predizione a 2 bit

- Loop da 10 ripetizioni : 90% corrette




```
Loop: 1st loop instr
      2nd loop instr
            .
            .
            .
      last loop instr
      bne $1,$2,Loop
      fall out instr
```

Predizione a 2 bit



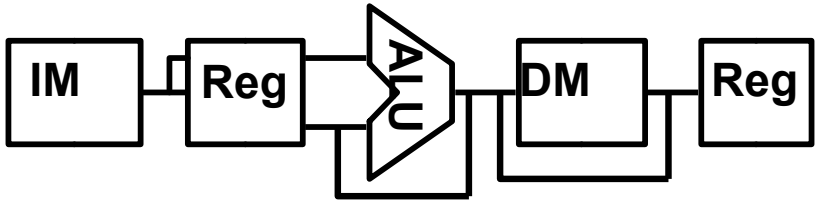

```
Loop: 1st loop instr
      2nd loop instr
            .
            .
            .
      last loop instr
      bne $1,$2,Loop
      fall out instr
```

- BHT contiene lo stato iniziale

Eccezioni

- Eccezioni (e interruzioni) equivalgono a conflitti sul controllo
  - R-type overflow
  - undefined instruction
  - Richiesta di I/O
  - Chiamata a Sistema Operativo (syscall)
  - Errore hardware
- Occorre fermare la pipeline
- Completare se possible istruzioni iniziate
- Registrare la causa dell'interruzione
- Salvare l'indirizzo dell'istruzione interrotta
- Saltare a un indirizzo prefissato per gestire problema (SO)

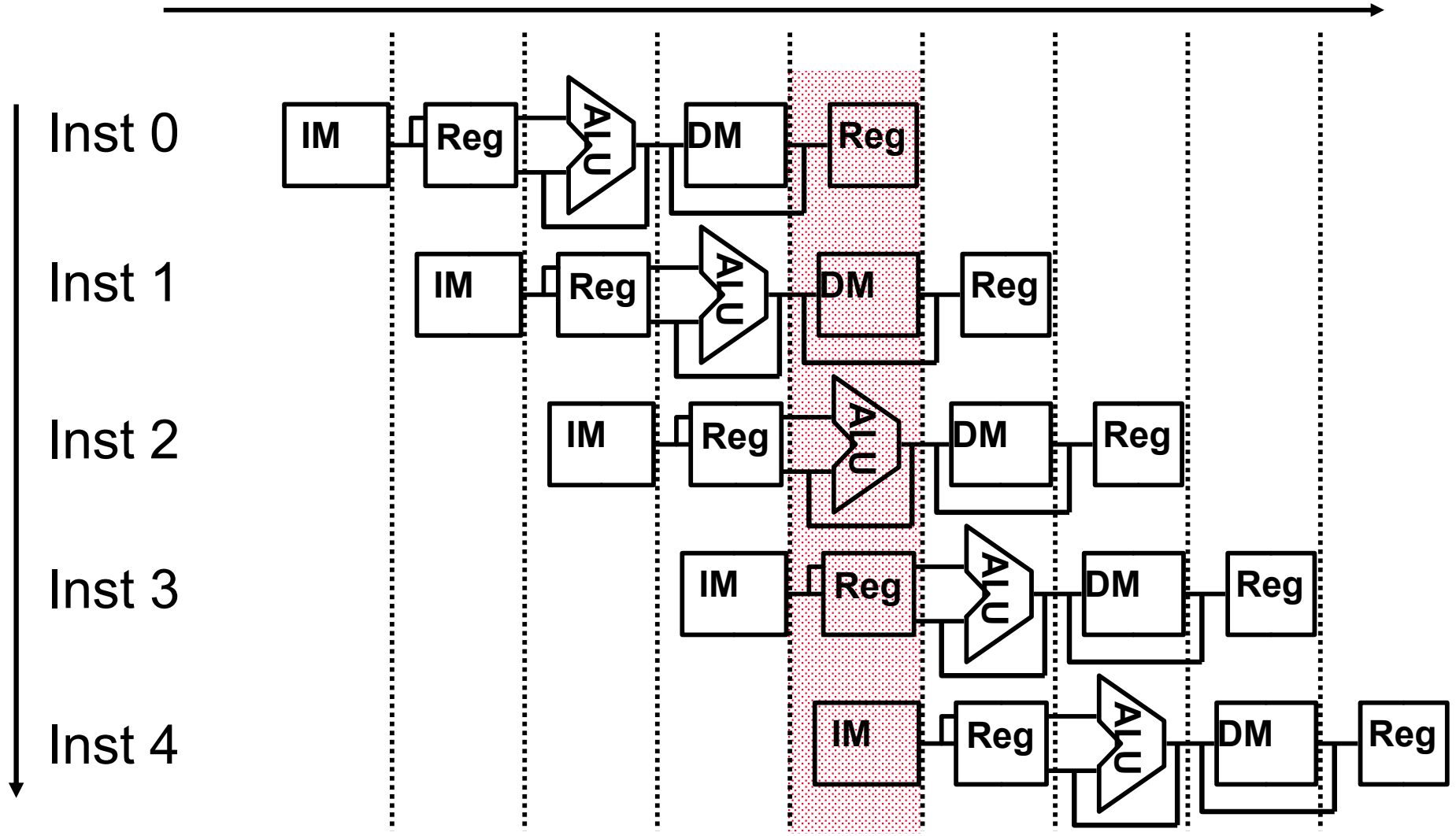
Fasi della pipeline ed eccezioni



	Fase ?	Sincrone ?
❑ Arithmetic overflow	EX	yes
❑ Undefined instruction	ID	yes
❑ TLB or page fault	IF, MEM	yes
❑ I/O service request	any	no
❑ Hardware malfunction	any	no

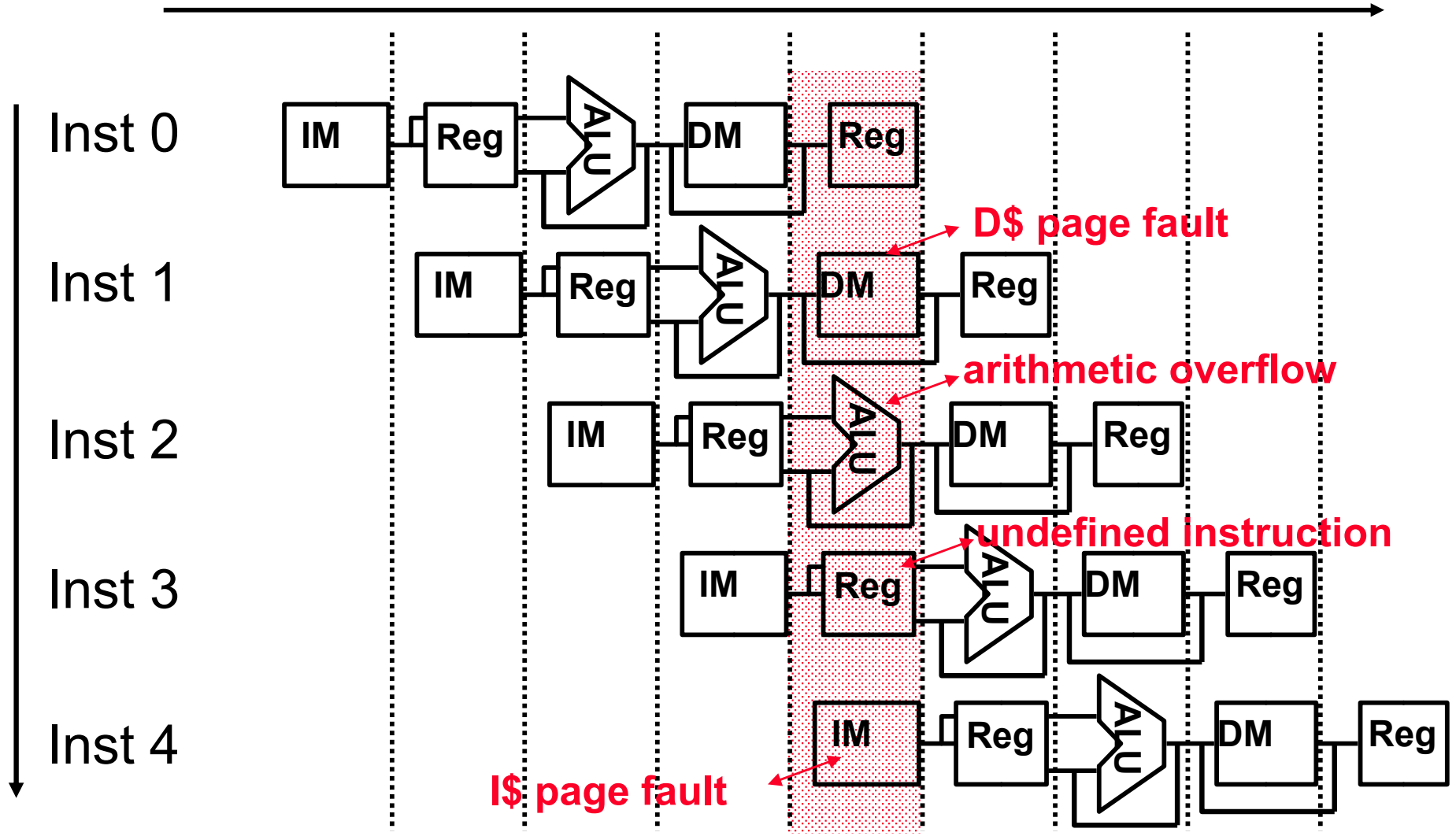
❑ Attenzione alle eccezioni multiple in *single* clock cycle

Eccezioni multiple simultanee



- Hardware deve dare priorità

Eccezioni multiple simultanee

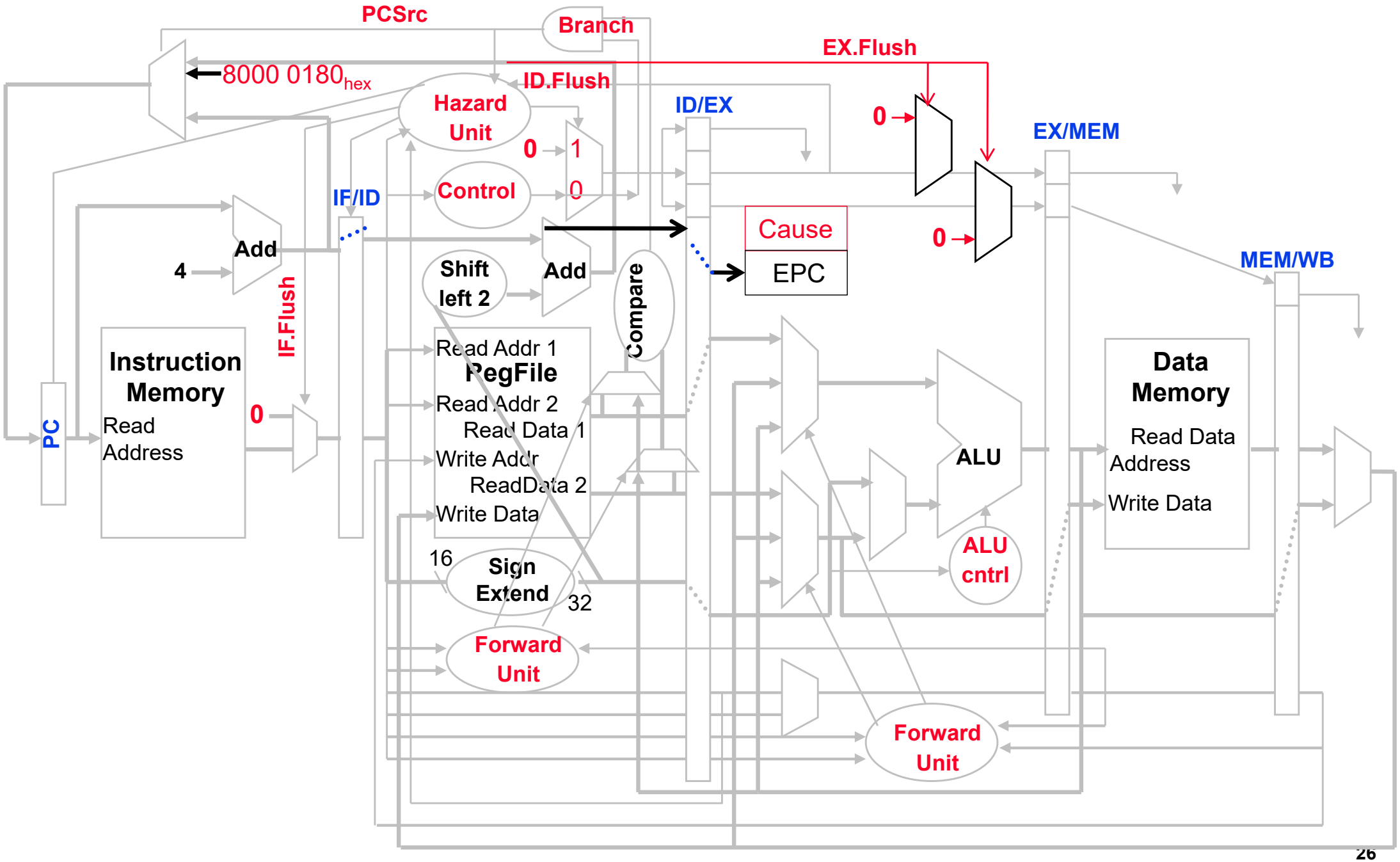


- Hardware deve dare priorità

Additions to MIPS to Handle Exceptions

- Cause register (records exceptions) – hardware to record in Cause the exceptions and a signal to control writes to it (`CauseWrite`)
- EPC register (records the addresses of the offending instructions) – hardware to record in EPC the address of the offending instruction and a signal to control writes to it (`EPCWrite`)
  - Exception software must match exception to instruction
- A way to load the PC with the address of the exception handler
  - Expand the PC input mux where the new input is hardwired to the exception handler address - (e.g., 8000 0180_{hex} for arithmetic overflow)
- A way to flush offending instruction and the ones that follow it

Datapath con gestione delle eccezioni



Summary

- All modern day processors use pipelining for performance (a CPI of 1 and fast a CC)
- Pipeline clock rate limited by slowest pipeline stage – so designing a balanced pipeline is important
- Must detect and resolve hazards
  - Structural hazards – resolved by designing the pipeline correctly
  - Data hazards
    - Stall (impacts CPI)
    - Forward (requires hardware support)
  - Control hazards – put the branch decision hardware in as early a stage in the pipeline as possible
    - Stall (impacts CPI)
    - Delay decision (requires compiler support)
    - Static and dynamic prediction (requires hardware support)
- Pipelining complicates exception handling